
Stochastic Penalty-Barrier Methods for Constrained Machine Learning

Adam Bosák

Artificial Intelligence Center,
CTU in Prague

Andrii Kliachkin

Artificial Intelligence Center,
CTU in Prague

Jana Lepšová

Artificial Intelligence Center,
CTU in Prague

Gilles Bareilles

CMAP, École Polytechnique,
Palaiseau, France

Jakub Mareček

Artificial Intelligence Center,
CTU in Prague

Abstract

Constrained machine learning enables fairness-aware training, physics-informed neural networks, and integration of symbolic domain knowledge into statistical models. Despite its practical importance, no general method exists for the non-convex, non-smooth, stochastic setting that arises naturally in deep learning. We propose the Stochastic Penalty-Barrier Method (SPBM), which extends classical penalty and barrier methods to this setting via exponential moving average of dual updates, a stabilized penalty schedule, and the Moreau envelope to handle non-smoothness. Experiments across multiple settings show that SPBM matches or outperforms existing constrained optimization baselines while incurring only linear runtime overhead compared to unconstrained Adam for up to 10,000 constraints.

1 Introduction

There has been much recent interest in constrained machine learning (CML) [1]. CML enables the enforcement of fairness constraints [2–4]. It also enables the implementation of domain-decomposition methods in physics-inspired neural networks [5], or mitigates known failure modes such as ill-conditioning [6]. Ultimately, CML could enable combining statistical machine learning with constraints from human-curated symbolic models [7]. One of the reasons of the success of CML is that it performs better than standard regularization; see Fig. 1 for a motivating example.

Yet, enforcing fairness constraints [3, 4] requires the constraints to be stochastic and non-convex. Examples include learning physics-inspired neural networks (PINNs) defined as a constrained optimization problem [8, 9], or any other user’s specified constraints dependent on the output of the neural network, such as causality of PINNs [7].

There is a substantial and recent body of work on CML. However, most current works assume simplified settings: Curtis et al. [10], Fang et al. [11] focus on deterministic constraints; Curtis et al. [12] consider simplified deterministic linear constraints; Necoara and Nedić [13], Necoara and Singh [14], Singh et al. [15], Singh and Necoara [16], Boob et al. [17] address stochastic constrained convex composite problems; finally, Stochastic Switching Subgradient (SSw) Huang and Lin [18] assumes stochastic non-convex and weakly convex constraints, whereas (SSL-ALM) Huang et al. [19] tackles stochastic non-convex objective with linear constraints. Although the survey on fairness [4] empirically shows that both SSw and SSL-ALM (despite not originally designed for) might also work in the constrained stochastic non-convex setting, *it remains unclear how to implement the training of neural networks subject to general stochastic, non-convex, non-smooth constraints efficiently.*

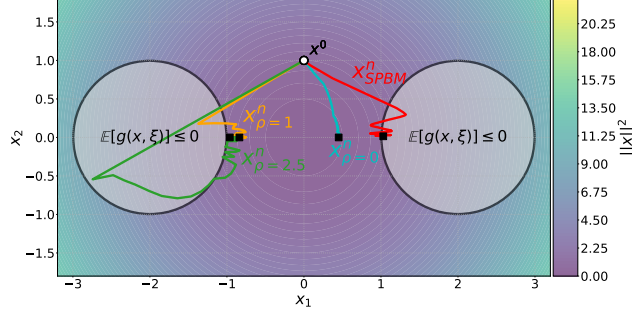


Figure 1: Motivation for using SPBM over standard regularization with the penalized objective $F_\rho(x) = f(x) + \rho \|\mathbb{E}[g(x, \xi)]\|^2$, with $f = \|x\|^2$ and a certain choice of g . For large ρ , SGD applied to $F_\rho(x)$ could be unstable, whereas for too small ρ , the solution produced by SGD applied to $F_\rho(x)$ could be infeasible. In contrast, SPBM is stable and converges to the feasible set; see Appendix A.

In this paper, we study the problem of optimizing a stochastic non-convex non-smooth objective subject to stochastic non-convex non-smooth constraints:

$$\min_{x \in \mathbb{R}^n} \mathbb{E}[f(x, \xi)] \quad \text{s.t.} \quad \mathbb{E}[g(x, \xi)] \leq 0, \quad (1)$$

where $\xi \sim \Xi$ is a random variable and $f : \mathbb{R}^n \times \Xi \rightarrow \mathbb{R}$ and $g : \mathbb{R}^n \times \Xi \rightarrow \mathbb{R}^m$ are also locally Lipschitz, which allows for them to be differentiable almost everywhere by Rademacher’s theorem. Note that the scenario where f is the loss function of a neural network falls into this setup, allowing to enforce fairness in neural network or solve PINNs as constrained optimization problems.

We approach the problem by introducing a new method called the Stochastic Penalty-Barrier Method (SPBM), an extension to the stochastic non-smooth non-convex setting of the original Penalty-Barrier Method (PBM) of Ben-Tal and Zibulevsky [20], which was introduced for convex deterministic problems. We enable the extension by utilizing the Moreau envelope from proximal methods [21] proven to be useful for non-smooth settings, together with standard techniques used in ML, such as exponential averaging for noise reduction and a careful choice of penalty updating function. SPBM and PBM are primal-dual methods designed around penalty-barrier functions which enable unification of the constraints. Both SPBM and PBM also utilize penalty parameters that enable a user to specify the speed of convergence towards the feasible region based on the given problem.

Interestingly, Ben-Tal and Zibulevsky [20] show that for a specific choice of dual update rule and penalty parameter update rule, PBM is equal to the augmented Lagrangian as defined in [22]. Hence, in some sense, SPBM is a generalization of SSL-ALM [19].

Our contributions. The contributions of this paper are as follows:

- We propose SPBM, a stochastic extension of the penalty–barrier method of [20].
- We empirically test SPBM on a range of fairness-constrained neural network problems, benchmarking against currently available approaches for solving (1). Our experiments include training robust classifiers on CIFAR-10 and CIFAR-100 datasets [23], fair classification tasks based on Folktables [24] and the Dutch census dataset [25], and training of physics-inspired neural networks based on the Helmholtz and viscous Burgers equations.
- Our open-source code can be used as a drop-in replacement for PyTorch optimizers training unconstrained neural networks, requiring only a few additional lines of code to specify the constraints. This integration allows existing publicly available models to be easily converted into more robust and safer versions.

Paper structure. The rest of the paper is organized as follows. After reviewing the related literature, we present preliminaries about the penalty/barrier method in Section 2. We introduce our stochastic extension of the penalty/barrier method in Section 3. Our experiments are described in Section 4 and analyzed in Section 5.

Related work. Historically quite recent, but pioneering work of Necoara and Nedić [13], Necoara and Singh [14], Singh et al. [15], Singh and Necoara [16] and Boob et al. [17] addresses functionally

constrained convex composite problems. This allows detailed analyses, but does not extend to the non-convex setting of problem (1).

Subsequently, Berahas et al. [26], Curtis et al. [27] introduced SQP-based methods for stochastic, non-convex objectives subject to deterministic equality constraints. Related SQP approaches under the same objective and constraint assumptions were studied in Fang et al. [28] and Na et al. [29]. Stochastic Ghost [30] could be seen as a continuation of this work, computing the update direction by solving a number of quadratic subproblems in each iteration. A similar approach was proposed in Ma et al. [31] for weakly-convex objectives and constraints, which solves a sequence of strongly convex subproblems formed with a quadratic regularization term. Stochastic non-convex optimization under deterministic non-convex inequality and/or equality constraints was also considered in Curtis et al. [32], Shi et al. [33], Na et al. [34]. Finally, SQP methods for problems with both stochastic objectives and stochastic constraints were proposed in Oztoprak et al. [35]. It is important to note these SQP methods require the computation and storage of a Hessian, which, in our case where f is a neural network, makes them impractical. Recently, some interior point methods have been proposed to solve non-convex constrained optimization, such as Curtis et al. [10] or Curtis et al. [12] and an SQP variant Fang et al. [11]. These methods, however, assume a strictly feasible initial point or deterministic constraints, making them unsuitable for our nonconvex stochastic constrained setting.

In a different direction, Bollapragada et al. [36] employed an augmented Lagrangian framework for stochastic objectives with deterministic linear constraints. An elaborate analysis by Huang et al. [19] considers stochastic non-convex objective and stochastic linear inequality constraints and a smoothed augmented Lagrangian (SSL-ALM). They establish the stochastic oracle complexity of their method. Cooper, the PyTorch framework for constrained deep learning [37], implements SSL-ALM without smoothing, while `humancompatible.train` [38] implements the smoothed version as well.

Stochastic Switching Subgradient (SSw) [18] is a method which extends the classical method of Polyak [39]. SSw addresses stochastic optimization problems with stochastic non-convex objective and stochastic weakly-convex or convex constraints. At each iteration, the method reduces the constrained problem to a sequence of subgradient updates with respect to the maximum constraint function g in the problem (1).

The penalty-barrier multiplier method, which serves as the basis for our stochastic extension, was introduced by Ben-Tal and Zibulevsky [20]. Later, it was generalized to solve deterministic non-convex non-linear problems in Kočvara and Stingl [40, 41, 42]. Recently, De Marchi and Themelis [43] defined a deterministic non-convex variant of problem (1) by combining a barrier term with an L_1 penalty into a new objective function. Our method is also closely related to the proximal augmented Lagrangian method, which has been studied since the 1970s [44–55].

2 Review of Penalty/Barrier Multiplier Method (PBM)

The method [20] was proposed for convex programming problems:

$$f^* = \inf\{f(x) : g_i(x) \leq 0, i = 1, \dots, m, x \in \mathbb{R}^n\}, \quad (2)$$

where $f, g_1, \dots, g_m$ are closed proper convex functions. Ben-Tal and Zibulevsky showed that PBM is closely related to the proximal point algorithm in the sense of the sequence of generated dual variables. One of the main ideas behind the method is the transformation of the constraints onto a uniform space using a strictly increasing twice differentiable strictly convex function φ with some additional properties; see [20, Section 3] for details. These properties imply that, provided a positive number p , the function $t \mapsto \varphi(t/p)$ is convex increasing and

$$p\varphi(t/p) \leq 0 \quad \text{if and only if} \quad t \leq 0. \quad (3)$$

Using (3), the original problem (2) can be rewritten to

$$f^* = \inf\{f(x) : p_i\varphi(g_i(x)/p_i) \leq 0, i = 1, \dots, m, x \in \mathbb{R}^n\}, \quad (4)$$

where p_i is referred to as the *penalty parameter* associated with the i -th constraint, and φ is referred to as the *penalty-barrier function*. The function $\mathcal{L} : \mathbb{R}^n \times \mathbb{R}_+^m \times \mathbb{R}_{++}^m \rightarrow \mathbb{R}$, referred to as the *Lagrangian* for problem (4), has the form $\mathcal{L}_p(x, \lambda) = f(x) + \sum_{i=1}^m \lambda_i p_i \varphi(g_i(x)/p_i)$ and it is used

in the following iterative scheme for solving (4):

$$\begin{aligned} x^{k+1} &= \arg \min_x \mathcal{L}_{p^k}(x, \lambda^k) \\ \lambda^{k+1} &= \lambda^k \odot \varphi' (g(x^{k+1}) \oslash p^k) \end{aligned} \quad (5)$$

$$p^{k+1} = \pi^k(\lambda^{k+1}) \quad (6)$$

where $\odot$ and $\oslash$ denotes the element-wise product and division, and φ' and π^k are applied coordinate-wise. The initial λ^0 is required to be positive and $\pi^k : \mathbb{R}_{++} \rightarrow \mathbb{R}_{++}$ is referred to as the *penalty updating function*. We recall two glued penalty/barrier functions: the *Quadratic-Logarithmic penalty/barrier function* φ_{QL} and the *Quadratic-Reciprocal penalty/barrier function* φ_{QR} :

$$\varphi_{\text{QL}}(t) = \begin{cases} t + \frac{1}{2}t^2 & \text{if } t \geq -\frac{1}{2}, \\ -\frac{1}{4}\log(-2t) - \frac{3}{8} & \text{if } t < -\frac{1}{2}, \end{cases} \quad \varphi_{\text{QR}}(t) = \begin{cases} t + \frac{1}{2}t^2 & \text{if } t \geq -\frac{1}{3}, \\ \frac{32}{27}(\frac{1}{1-t}) - \frac{7}{6} & \text{if } t < -\frac{1}{3}. \end{cases}$$

The penalty updating functions in Ben-Tal and Zibulevsky [20] are chosen as

$$\pi^k(t) = \pi_0(\kappa)^k t \quad \text{or} \quad \pi^k(t) = \pi_0(\kappa)^k, \quad (7)$$

where they set $\kappa \in (0, 1)$ and $\pi_0 > 0$ typically between 10 – 1000; see the original paper for details.

3 Stochastic Penalty/Barrier Method (SPBM)

Our method, Stochastic Penalty-Barrier Method (SPBM), extends the penalty and barrier framework of Ben-Tal and Zibulevsky [20] to the stochastic setting. While the structure follows their iterative update rules, the extension is non-trivial and requires three key modifications. First, since expectations are approximated via mini-batching, we apply exponential averaging to the dual updates to reduce noise. Second, the penalty parameters are updated differently from Ben-Tal and Zibulevsky [20] to ensure stability. Third, since the resulting objective is non-smooth, we perform an inexact proximal point update to recover a tractable minimization step. In the following, we present the mini-batching notation and address the main challenges in detail. The full algorithm is presented in Alg. 1.

Mini-batching. We use mini-batching with batch size $B \in \mathbb{N}$ to sample both the objective distribution and the constraint distribution. A mini-batch is a set $\{\xi_j\}_{j=1}^B$ of independent and identically distributed random variables $\xi_1, \dots, \xi_B \stackrel{iid}{\sim} \Xi$. Stochastic estimates of the objective and constraint functions are computed from the mini-batch as follows:

$$\bar{f}(x) = \frac{1}{B} \sum_{j=1}^B f(x, \xi_j), \quad \bar{g}(x) = \frac{1}{B} \sum_{j=1}^B g(x, \xi_j).$$

Similarly, we derive a stochastic estimate of the the Lagrangian $\mathcal{L}_p(x, \lambda)$ and its gradient:

$$\begin{aligned} \bar{\mathcal{L}}_p(x, \lambda) &= \frac{1}{B} \sum_{j=1}^B \left(f(x, \xi_j) + \sum_{i=1}^m \lambda_i \varphi(g_i(x, \xi_j)/p_i) \right), \\ \bar{\nabla} \mathcal{L}_p(x, \lambda) &= \frac{1}{B} \sum_{j=1}^B \left(\nabla f(x, \xi_j) + \sum_{i=1}^m \lambda_i \varphi'(g_i(x, \xi_j)/p_i) \nabla g_i(x, \xi_j) \right). \end{aligned}$$

Challenge 1: Non-smoothness. The intuition is to optimize over the approximate solution of the Moreau envelope of the Lagrangian, with given p^{k+1} , s^k and λ^{k+1} :

$$\min_x \left(\bar{\mathcal{L}}_{p^{k+1}}(x, \lambda^{k+1}) + \frac{\mu}{2} \|x - s^k\|^2 \right).$$

In theory, the Moreau envelope of a weakly-convex non-smooth function shares the same minimizers while yielding a smooth approximation provided that the parameter μ is chosen sufficiently large [56]. In practice, we find an approximate minimizer of the Moreau envelope by performing a single step of Adam with respect to x (OneStepAdam at line 7). This is the same as performing an inexact proximal point step on function $\frac{1}{\mu} \bar{\mathcal{L}}_{p^{k+1}}$ (i.e. $x^{k+1} \approx \text{prox}_{\frac{1}{\mu} \bar{\mathcal{L}}_{p^{k+1}}(\cdot, \lambda^{k+1})}(s^k)$). We then translate the argument s of the proximal operator through the exponential averaging (line 8, as done in Parikh et al. [21]). Recently, Zhang and Luo [54] used a similar idea in the proximal version of the augmented Lagrangian for a deterministic non-convex setting with linear constraints. Its stochastic counterpart with linear constraints was introduced in [19].

Algorithm 1 Stochastic Penalty/Barrier Method (SPBM)

```
1: Input: Initial primal variable  $x^0 \in \mathbb{R}^n$ , initial dual variable  $\lambda^0 \in \mathbb{R}_{++}^m$ , initial penalty parameter  $p^0$  as all-ones vector, learning rate  $\alpha > 0$ , exponential decay rates  $\beta_1 \approx 0.9$ ,  $\beta_2 \approx 0.999$ , mini-batch size  $B \in \mathbb{N}$ ,  $\gamma \in [0, 1)$ ,  $\mu \geq 0$ ,  $\delta \in [0, 1)$ ,  $s^0 = x^0$ , and  $\hat{\pi} \in \{\text{Id}, \hat{\pi}_{\text{adaptive}}\}$ .
2: for  $k = 1$  to  $K$  do
3:   Sample  $\xi_1, \xi_2, \dots, \xi_B \stackrel{iid}{\sim} \Xi$ 
4:    $\lambda^{k+1} = \gamma \lambda^k + (1 - \gamma) \lambda^k \odot \varphi'(\bar{g}(x^k) \odot p^k)$                                 {▷ Dual variable update}
5:    $p^{k+1} = \hat{\pi}(\bar{g}(x^k), p^k)$                                                     {▷ Penalty parameter update}
6:   Generate  $y^k = \nabla \mathcal{L}_{p^{k+1}}(x^k, \lambda^{k+1}) + \mu(x^k - s^k)$ 
7:    $x^{k+1} = \text{OneStepAdam}(x^k, y^k, \alpha, \beta_1, \beta_2)$                                 {▷ Primal variable update}
8:    $s^{k+1} = \delta s^k + (1 - \delta)x^{k+1}$                                               {▷ Prox-center update}
9: end for
```

Challenge 2: Mini-batching and penalty updating functions. Some constraints (e.g., fairness constraints) are defined over subpopulations of the dataset, such as those induced by sensitive attributes (e.g., sex) or class labels in image classification. Evaluating such constraints requires at least one sample from each subpopulation. To ensure this, we construct each mini-batch to include an equal number of samples from each subpopulation.

Small mini-batch sizes can cause the stochastic estimates of each constraint to be heavily influenced by outliers. Simultaneously, using the penalty updating functions (7) in the iterative scheme (6) imply that eventually $p \rightarrow 0$. This makes the updating rule (5) unstable. Empirically, setting p to the all-ones vector worked well in most of our experiments. In more complicated cases, where the convergence toward the feasible region was too slow, we used the following method $\hat{\pi} : \mathbb{R}^m \times \mathbb{R}^m \rightarrow \mathbb{R}^m$:

$$\hat{\pi}_{\text{adaptive}}(g(x), p) = \text{Clip}_{[0.1, 1]} \left[\left(K + \frac{(1 - K)}{\varphi'(g(x)) + \varepsilon} \right) p \right], \quad (8)$$

where all operations (sum, product, division) are performed coordinate-wise, we set $K \in (0, 1)$, $\varepsilon = 10^{-8}$ and $\text{Clip}_{[0.1, 1]}$ denotes the coordinate-wise clipping on interval $[0.1, 1]$.

The motivation behind the formula (8) is the same as the decrease in p in Eq. (6). The smaller p_i , the bigger is the update of the dual variable λ_i of the non-satisfied constraints and the smaller is the update of duals for satisfied constraints; see Eq. (5). For this reason, one might want to decrease the penalties only for unsatisfied constraints (recall that there is exactly one penalty parameter for each constraint). The formula (8) decreases or increases p_i based on the current satisfaction of the i -th constraint. Such “adaptive” decrease and increase is controlled by the derivative of the penalty/barrier, i.e., for $g(x) > 0$ we get $\varphi'(g(x)) > 1$, ultimately decreasing the penalty parameter p (and vice versa for $g(x) < 0$).

4 Numerical Experiments

To evaluate SPBM against state-of-the-art methods for CML, we create a benchmark comparing SPBM to three other optimization methods: Adam [57] (with no constraints), SSw [18], and SSL-ALM [19].¹ Our benchmark contains 6 classification problems based on the ACSIncome dataset, the Dutch Census dataset, and the CIFAR-10 and CIFAR-100 datasets, and two experiments involving Physics-Informed Neural Networks (PINNs). We use Adam as a baseline for the classification problems without constraints. For PINNs, we use Adam to optimize the network as suggested in the original paper [58] with regularization term of the initial and boundary constraints as a hyperparameter. We omit SSw in PINNs benchmarks due to its poor performance on previous experiments.

Datasets and problem tasks. Our fairness-based experiments are based on three datasets. For the ACSIncome dataset [24], we use a subset corresponding to the state of Virginia, which includes 46,144 samples, and consider the binary classification task of predicting whether an individual’s income exceeds \$50,000. For the Dutch Census dataset [25], which represents aggregated groups of

¹We choose not to include the Stochastic Ghost [30] based on the benchmarking paper [4], in which SSL-ALM outperforms it on all tasks.

people in the Netherlands and includes 60,000 samples, the binary classification task is to predict whether an individual has a high-level or low-level profession. The CIFAR-10 dataset [23] contains 60,000 32x32 color images in 10 different classes, whereas the CIFAR-100 dataset contains 100 different classes with 600 images per class. For both datasets, the task is to learn an accurate classifier of all ten classes. PINNs are data efficient, allowing us to sample as many data as needed for the two PINNs experiments. We sample 2601 training data points, and import from Son et al. [8] 1000 validation samples, 1000 test samples of each solution of the differential equation. The task is to solve the differential equation in both PINNs experiments.

Fairness as constrained optimization. In classification experiments, we implement three types of constraints. The first type is deterministic and it consists in enforcing the weights of the neural network to lie inside a ball:

$$\|W_i\|_F \leq 2, \quad \text{for each layer } i, \quad (9)$$

where W_i is a weight matrix of the fully connected neural network and $\|\cdot\|_F$ is the Frobenius norm. The remaining two types of constraints are stochastic and their goal is to enforce a similar prediction accuracy across different groups – either demographic groups in datasets based on demographic data or classes in the CIFAR datasets. The first approach, introduced and implemented as part of the `fairret` package [59], aggregates violations of each group from the set of groups $\mathcal{R}$ into a single value using the L_1 -norm, and thus results in just one constraint ($m = 1$):

$$\sum_{R_i \in \mathcal{R}} |\mathbb{E}_{\xi \sim \mathcal{D}[\text{group } R_i]}[f(x, \xi)] - \mathbb{E}[f(x, \xi)]| \leq \epsilon, \quad (10)$$

where $\mathcal{D}[\text{group } R_i]$ is the distribution over the group R_i . The second approach is to consider all pairwise differences between groups. That is, we enforce for all $R_i, R_j \in \mathcal{R}$ such that $i \neq j$:

$$|\mathbb{E}_{\xi \sim \mathcal{D}[\text{group } R_i]}[f(x, \xi)] - \mathbb{E}_{\xi \sim \mathcal{D}[\text{group } R_j]}[f(x, \xi)]| \leq \epsilon, \quad \text{for every } i \neq j, \quad (11)$$

which results in $m = |\mathcal{R}| \cdot (|\mathcal{R}| - 1)$ constraints.

Solving PDEs as constrained optimization. PINNs are neural networks trained to satisfy physical laws governed by nonlinear partial differential equations (PDEs). An example of such a physical law can be the Helmholtz equation:

$$\Delta u + u = q(z_1, z_2), \quad \text{for every } (z_1, z_2) \in \Omega, \quad (12)$$

$$u(z_1, z_2) = 0, \quad \text{for every } (z_1, z_2) \in \partial\Omega, \quad (13)$$

where Ω is the square $\Omega = [-1, 1]^2$, $\partial\Omega$ denotes its boundary and the function $q(z_1, z_2)$ is fixed (see Appendix B for details). Since boundary conditions are known to cause instability in PINN training [60, 8, 61], it is natural to enforce them as constraints, yielding the constrained optimization problem:

$$\min_x \mathbb{E}_{\xi \sim \mathcal{D}[\Omega]} [|Nu_{\text{nn}}(x, \xi) - h_1(\xi)|^2] \quad (14)$$

$$\text{s.t. } \mathbb{E}_{\xi \sim \mathcal{D}[\partial\Omega]} [|Tu_{\text{nn}}(x, \xi) - h_2(\xi)|^2] \leq \epsilon_{\text{PINN}} \quad (15)$$

where $\epsilon_{\text{PINN}} = 10^{-4}$, N is a differential operator, T is a trace operator, u_{nn} is a neural network representing the solution of the PDE with weights x and collocation points ξ . Functions h_1 and h_2 are prescribed and $\Omega \subseteq \mathbb{R}^d$ is a bounded set. Including the time variable t as an extra element of ξ , we can use (14) and (15) even for time-dependent problems. This allows the constraint (15) to include initial value conditions. Note that the problem defined by (14) and (15) is an instance of (1) with $f := |Nu_{\text{nn}}(x, \xi) - h_1(\xi)|^2$ and $g := |Tu_{\text{nn}}(x, \xi) - h_2(\xi)|^2$.

Experiments. We denote our experiments by (E₁)–(E₈) and summarize them in Table 1. Note that in (E₂) the groups are based on the SEX attribute, in (E₃) the six groups are based on the cartesian product of the attributes of SEX and MARRIAGE STATUS (Divorced, Married, Single), and in (E₄) there are 18 groups based on the cartesian product of the SEX and AGE attributes. The experiments (E₇) and (E₈) are related to PINNs; namely, (E₇) is based on the Helmholtz equation and (E₈) is based on the viscous Burgers equation; see Appendix B for details on the differential equations.

Experimental setup. Details on hyperparameters, network architectures and data processing are listed in Appendix C.

Table 1: Summary of experiments.

Exp.	Dataset	Experiment type	Groups $\mathcal{R}$	Constraint type	# Constraints (m)
(E ₁)	ACSIncome	binary classification	–	(9)	6
(E ₂)	ACSIncome	binary classification	2 (sex)	(10)	1
(E ₃)	ACSIncome	binary classification	6 (sex $\times$ marital)	(11)	30
(E ₄)	Dutch Census	binary classification	18 (sex $\times$ age)	(11)	306
(E ₅)	CIFAR-10	multiclass classif.	10 (classes)	(11)	90
(E ₆)	CIFAR-100	multiclass classif.	100 (classes)	(11)	9900
(E ₇)	Sampled $[-1, 1]^2$	interpolation of (14) with (12)	–	(15) with (13)	1
(E ₈)	Sampled $[0, 1] \times [-1, 1]$	interpolation of (14) with (17)	–	(15) with (18), (19)	2

5 Analysis of the Experiments

In this section, we analyze the experiments presented in Section 4. All plots show the mean across seeds (solid curve) and $\pm$ one standard deviation (shaded area). We present in the main paper experiments (E₄), (E₆), (E₇) and (E₈) that enable a constructive assessment of the methods.

Fairness analysis. In experiment (E₄) shown in Fig. 2, SSL-ALM and SPBM reach the same test loss with the same convergence rate while both satisfying the test constraints, hence have equally good performance. On the other hand, SSw is much worse in both criteria.

In experiment (E₆) shown in Fig. 3, which has the highest number of constraints in our benchmark. SPBM reaches a much better test and train loss compared to SSL-ALM while satisfying the train constraints. Although SSL-ALM satisfies test constraints compared to SPBM, SSL-ALM fails to minimize the loss function. Note that SPBM also achieves the same test loss as Adam while violating the constraints less. SSw has poor results with regards to the loss minimization.²

Regarding experiments (E₁)–(E₃) and (E₅) shown in Appendix E, both SPBM and SSL-ALM perform the same with regards to the test loss convergence and test constraint satisfaction, while SSw performed the worst on all experiments except for (E₁).

PINNs analysis (E₇)–(E₈). It is evident from Fig. 4, Fig. 5 and Table 4 that SPBM outperforms both SSL-ALM and Adam with regularization in both the constraint satisfaction and in terms of the quality of the solution found, which is assessed by the test loss. This might be due to a better satisfaction of the initial conditions and the boundary conditions compared to the SSL-ALM and Adam counterparts, which can be critical for the solution quality [60, 8, 61].

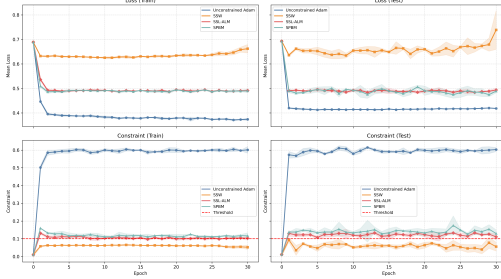


Figure 2: (E₄): Dutch, Demographic Parity, Pairwise, $m = 306$: mean loss (top row: train and test) and mean largest constraint (bottom row: train and test) values over 3 runs of 30 epochs of each method with random parameter initialization. The shaded region corresponds to ± 1 standard deviations. The red dotted line corresponds to the constraint threshold.

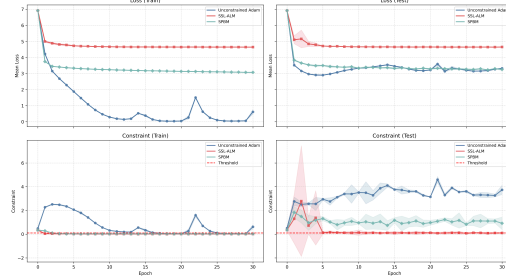


Figure 3: (E₆): CIFAR-100, Equal Accuracy, Pairwise, $m = 9900$: mean loss (top row: train and test) and mean largest constraint (bottom row: train and test) values over 3 runs of 30 epochs of each method with random parameter initialization. The shaded region corresponds to ± 1 standard deviations. The red dotted line corresponds to the constraint threshold.

Runtimes. We report the average time per epoch for (E₅) and (E₆) in Table 2. Runtimes for the other experiments are omitted, as their small scale makes them sensitive to hardware variability.

²SSw distorts the plots slightly, thus we upload two versions: one without SSw and the other with SSw in Appendix E, see Fig. 10.

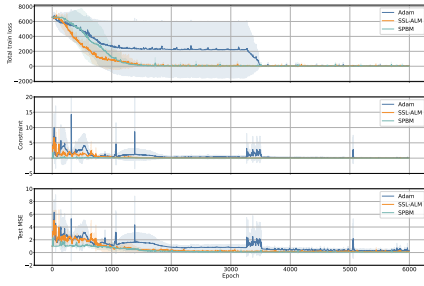


Figure 4: (**E**₇): **Helmholtz PDE, PINN, $m = 1$:** **mean PINN loss** defined as a sum of loss and constraints as presented in [58] (top row), **mean constraint** (middle row), **mean test loss** which assesses solution quality (bottom row). Values over 3 runs of 6000 epochs of each method with random parameter initialization. The shaded region corresponds to ± 1 standard deviations.

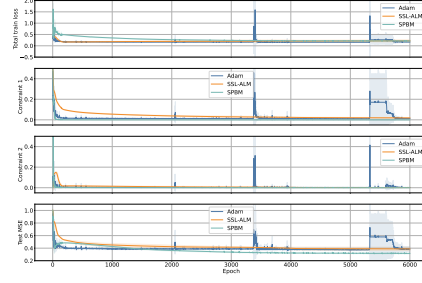


Figure 5: (**E**₈): **Viscous Burgers PDE, PINN, $m = 2$:** **mean PINN loss** defined as a sum of loss and constraints as presented in [58] (top row), **mean constraints** (2 middle rows), **mean test loss** which assesses solution quality (bottom row). Values over 3 runs of 6000 epochs of each method with random parameter initialization. The shaded region corresponds to ± 1 standard deviations.

Unconstrained Adam is the fastest, followed by SSw, SSL-ALM, and SPBM. It is clear that our effective implementation makes big-scale training feasible with a linear scaling compared to the unconstrained minimization (only up to a factor of 3 compared to Adam).

Table 2: **Runtimes (s)** per epoch per algorithm on the hardware described in Section 4 for (**E**₅) and (**E**₆). The experiments were repeated 3 times with different seeds.

	m	ADAM	SSW	SSL-ALM	SPBM (OURS)
(E ₅)	90	0.86 ± 0.01	1.97 ± 0.01	1.94 ± 0.01	2.77 ± 0.01
(E ₆)	9900	1.92 ± 0.01	2.4 ± 0.01	4.0 ± 0.01	5.86 ± 0.02

General take-aways. Based on the experiments, we can conclude with the following take-aways; our main comparison factor is the test loss and test constraints convergence of the methods:

- (1) *SPBM is competitive with SSL-ALM.* SPBM outperforms SSL-ALM in 3/8 experiments and matches it in the remaining 5/8. This suggests SPBM is a reliable alternative across problem settings.
- (2) *Dedicated multi-constraint primal-dual methods outperform single-constraint primal-only one.* SSw underperforms both SPBM and SSL-ALM in most experiments. We attribute this primarily to SSw’s limitation of handling only one constraint at a time, making it ill-suited for the multi-constraint settings considered here.
- (3) *Constrained training is computationally feasible.* Our implementation achieves only linear overhead relative to unconstrained optimization, demonstrating that training neural networks subject to fairness or user-specified constraints need not come at prohibitive computational cost.

6 Conclusion

We have presented SPBM, an extension of the penalty and barrier framework to non-convex, non-smooth, stochastic optimization, with three key modifications: exponential averaging of dual updates, a stabilized penalty parameter schedule, and the use of the Moreau envelope to address non-smoothness. Experimental results demonstrate that SPBM is competitive with or outperforms current state-of-the-art methods, while maintaining linear computational overhead over unconstrained optimization.

Table 3: Comparison of Adam, SSL-ALM, SSw and SPBM on experiments (E₁)–(E₆). We report the best test loss, together with the corresponding maximum and mean constraint violations (averaged over runs). For all metrics, smaller is better. Provided that the values differ, the best value is in bold and the second-best in brown.

Exp.	Method	Best loss	Epoch	Mean constraint	Max constraint
(E ₁)	Adam	0.41 ± 0.00	2	2.522 ± 0.062	9.807 ± 0.269
	SSL-ALM	0.41 ± 0.01	16	1.754 ± 0.012	2.000 ± 0.002
	SPBM	0.41 ± 0.00	4	1.375 ± 0.018	2.018 ± 0.037
	SSw	0.50 ± 0.01	2	1.173 ± 0.023	1.923 ± 0.018
(E ₂)	Adam	0.42 ± 0.00	1	0.225 ± 0.006	0.225 ± 0.006
	SSL-ALM	0.42 ± 0.00	3	0.145 ± 0.002	0.145 ± 0.002
	SPBM	0.43 ± 0.00	3	0.111 ± 0.013	0.111 ± 0.013
	SSw	0.43 ± 0.01	7	0.112 ± 0.010	0.112 ± 0.010
(E ₃)	Adam	0.42 ± 0.00	30	0.000 ± 0.000	0.369 ± 0.008
	SSL-ALM	0.51 ± 0.01	27	0.000 ± 0.000	0.109 ± 0.006
	SPBM	0.52 ± 0.01	10	0.000 ± 0.000	0.109 ± 0.019
	SSW	0.57 ± 0.00	22	0.000 ± 0.000	0.107 ± 0.004
(E ₄)	Adam	0.41 ± 0.00	5	0.000 ± 0.000	0.595 ± 0.009
	SSL-ALM	0.48 ± 0.00	12	0.000 ± 0.000	0.134 ± 0.007
	SPBM	0.47 ± 0.01	24	0.000 ± 0.000	0.183 ± 0.040
	SSw	0.63 ± 0.01	9	0.000 ± 0.000	0.070 ± 0.006
(E ₅)	Adam	1.10 ± 0.04	11	0.000 ± 0.000	1.027 ± 0.180
	SSL-ALM	1.13 ± 0.01	12	0.000 ± 0.000	0.634 ± 0.041
	SPBM	1.13 ± 0.02	11	0.000 ± 0.000	0.782 ± 0.150
	SSw	2.11 ± 0.06	6	0.000 ± 0.000	0.153 ± 0.113
(E ₆)	Adam	2.90 ± 0.03	5	0.000 ± 0.000	3.127 ± 0.291
	SSL-ALM	4.64 ± 0.00	29	0.000 ± 0.000	0.176 ± 0.032
	SPBM	3.23 ± 0.01	28	0.000 ± 0.000	1.471 ± 0.275
	SSw	4.82 ± 0.06	30	0.000 ± 0.000	0.143 ± 0.008

Table 4: Comparison of Adam, SSL-ALM, and SPBM on experiments (E₇) and (E₈). We report the best test loss, together with the corresponding constraint violations (averaged over runs).

Exp.	Method	Best loss	Epoch	Constraint 1	Constraint 2
(E ₇)	Adam	0.25 ± 0.2	5985	0.075 ± 0.086	∅
	SSL-ALM	0.1 ± 0.5	5908	0.013 ± 0.01	∅
	SPBM	0.04 ± 0.1	4668	0.003 ± 0.001	∅
(E ₈)	Adam	0.365 ± 0.01	3949	0.012 ± 0.0	0.002 ± 0.0
	SSL-ALM	0.389 ± 0.02	5999	0.02 ± 0.003	0.002 ± 0.0
	SPBM	0.315 ± 0.004	5306	0.0 ± 0.0	0.0 ± 0.0

Limitations. Our evaluation is limited to the fairness constrained optimization and PINNs. Moreover, convergence guarantees remain an open question in the fully stochastic non-convex non-smooth setting. The choice of penalty schedule also introduces hyperparameters that may require tuning.

Broader Impact. This work proposes a general constrained optimization method applicable to a range of settings, including fairness-aware classification. On the positive side, it enables practitioners to enforce fairness constraints during training, reducing discriminatory outcomes in deployed models. On the negative side, the flexibility of constraint specification could in principle be misused to encode constraints that entrench rather than mitigate bias, depending on how constraints are defined by the practitioner.

Acknowledgement The authors acknowledge the support of National Recovery Plan funded project MPO 60273/24/21300/21000 CEDMO 2.0 NPO. This work was supported in part by the European Union’s Horizon Europe research and innovation programme under grant agreement 101084642 (Codiet).

References

- [1] Juan Ramirez, Meraj Hashemizadeh, and Simon Lacoste-Julien. Position: Adopt Constraints Over Penalties in Deep Learning, July 2025. Cited on page 1.
- [2] Zhu Li, Adrián Pérez-Suay, Gustau Camps-Valls, and Dino Sejdinovic. Kernel dependence regularizers and Gaussian processes with applications to algorithmic fairness. *Pattern Recognition*, 132:108922, December 2022. ISSN 0031-3203. doi: 10.1016/j.patcog.2022.108922. Cited on page 1.
- [3] Maarten Buyl, Marybeth DeFrance, and Tijl De Bie. fairret: a framework for differentiable fairness regularization terms. In *International Conference on Learning Representations*, 2024. Cited on page 1.
- [4] Andrii Kliachkin, Jana Lepšová, Gilles Bareilles, and Jakub Marecek. Benchmarking stochastic approximation algorithms for fairness-constrained training of deep neural networks. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=JxmjzC6syB>. Cited on pages 1, 5, and 25.
- [5] Lu Lu, Raphael Pestourie, Wenjie Yao, Zhicheng Wang, Francesc Verdugo, and Steven G Johnson. Physics-informed neural networks with hard constraints for inverse design. *SIAM Journal on Scientific Computing*, 43(6):B1105–B1132, 2021. Cited on page 1.
- [6] Aditi Krishnapriyan, Amir Gholami, Shandian Zhe, Robert Kirby, and Michael W Mahoney. Characterizing possible failure modes in physics-informed neural networks. *Advances in neural information processing systems*, 34:26548–26560, 2021. Cited on page 1.
- [7] Sifan Wang, Shyam Sankaran, and Paris Perdikaris. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 421: 116813, 2024. Cited on page 1.
- [8] Hwijae Son, Sung Woong Cho, and Hyung Ju Hwang. Enhanced physics-informed neural networks with augmented lagrangian relaxation method (al-pinns). *Neurocomputing*, 548: 126424, 2023. Cited on pages 1, 6, 7, and 15.
- [9] Yongcun Song, Xiaoming Yuan, and Hangrui Yue. The admm-pinns algorithmic framework for nonsmooth pde-constrained optimization: a deep learning approach. *SIAM Journal on Scientific Computing*, 46(6):C659–C687, 2024. Cited on page 1.
- [10] Frank E Curtis, Xin Jiang, and Qi Wang. A single-loop stochastic feasible interior-point algorithm for nonlinear inequality-constrained optimization: F. curtis et al. *Mathematical Programming*, pages 1–38, 2026. Cited on pages 1 and 3.
- [11] Yuchen Fang, Jihun Kim, Sen Na, James Demmel, and Javad Lavaei. A trust-region interior-point stochastic sequential quadratic programming method. *arXiv preprint arXiv:2603.10230*, 2026. Cited on pages 1 and 3.
- [12] Frank E Curtis, Vyacheslav Kungurtsev, Daniel P Robinson, and Qi Wang. A stochastic-gradient-based interior-point algorithm for solving smooth bound-constrained optimization problems. *SIAM Journal on Optimization*, 35(2):1030–1059, 2025. Cited on pages 1 and 3.
- [13] Ion Necoara and Angelia Nedić. Minibatch stochastic subgradient-based projection algorithms for feasibility problems with convex inequalities. *Computational Optimization and Applications*, 80(1):121–152, 2021. Cited on pages 1 and 2.
- [14] Ion Necoara and Nitesh Kumar Singh. Stochastic subgradient for composite convex optimization with functional constraints. *Journal of Machine Learning Research*, 23(265):1–35, 2022. Cited on pages 1 and 2.

- [15] Nitesh Kumar Singh, Ion Necoara, and Vyacheslav Kungurtsev. Mini-batch stochastic subgradient for functional constrained optimization. *Optimization*, 73(7):2159–2185, 2024. Cited on pages 1 and 2.
- [16] Nitesh Kumar Singh and Ion Necoara. Stochastic halfspace approximation method for convex optimization with nonsmooth functional constraints. *IEEE Transactions on Automatic Control*, 2024. Cited on pages 1 and 2.
- [17] Digvijay Boob, Qi Deng, and Guanghui Lan. Stochastic first-order methods for convex and nonconvex functional constrained optimization. *Mathematical Programming*, 197(1):215–279, 2023. Cited on pages 1 and 2.
- [18] Yankun Huang and Qihang Lin. Oracle complexity of single-loop switching subgradient methods for non-smooth weakly convex functional constrained optimization. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 61327–61340. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/c132c02176577c4319a878f6417a331a-Paper-Conference.pdf. Cited on pages 1, 3, and 5.
- [19] Ruichuan Huang, Jiawei Zhang, and Ahmet Alacaoglu. Stochastic smoothed primal-dual algorithms for nonconvex optimization with linear inequality constraints. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 26106–26142. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/huang25au.html>. Cited on pages 1, 2, 3, 4, and 5.
- [20] Aharon Ben-Tal and Michael Zibulevsky. Penalty/barrier multiplier methods for convex programming problems. *SIAM Journal on Optimization*, 7(2):347–366, 1997. Cited on pages 2, 3, and 4.
- [21] Neal Parikh, Stephen Boyd, et al. Proximal algorithms. *Foundations and trends® in Optimization*, 1(3):127–239, 2014. Cited on pages 2 and 4.
- [22] R Rockafellar. Convex analysis. *Princeton Mathematical Series*, 28, 1970. Cited on page 2.
- [23] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images.(2009), 2009. Cited on pages 2 and 6.
- [24] Frances Ding, Moritz Hardt, John Miller, and Ludwig Schmidt. Retiring adult: New datasets for fair machine learning. *Advances in Neural Information Processing Systems*, 34, 2021. Cited on pages 2, 5, and 25.
- [25] Paul Van der Laan. *The 2001 Census in the Netherlands: Integration of Registers and Surveys*, pages 39–52. 12 2001. Cited on pages 2, 5, and 25.
- [26] Albert S. Berahas, Frank E. Curtis, Michael J. O’Neill, and Daniel P. Robinson. A stochastic sequential quadratic optimization algorithm for nonlinear equality constrained optimization with rank-deficient jacobians, 2023. URL <https://arxiv.org/abs/2106.13015>. Cited on page 3.
- [27] Frank E. Curtis, Michael J. O’Neill, and Daniel P. Robinson. Worst-case complexity of an sqp method for nonlinear equality constrained stochastic optimization. *Mathematical Programming*, 205(1):431–483, May 2024. ISSN 1436-4646. doi: 10.1007/s10107-023-01981-1. URL <https://doi.org/10.1007/s10107-023-01981-1>. Cited on page 3.
- [28] Yuchen Fang, Sen Na, Michael W. Mahoney, and Mladen Kolar. Fully stochastic trust-region sequential quadratic programming for equality-constrained optimization problems. *SIAM Journal on Optimization*, 34(2):2007–2037, 2024. doi: 10.1137/22M1537862. URL <https://doi.org/10.1137/22M1537862>. Cited on page 3.

- [29] Sen Na, Mihai Anitescu, and Mladen Kolar. An adaptive stochastic sequential quadratic programming with differentiable exact augmented lagrangians. *Mathematical Programming*, 199(1):721–791, May 2023. doi: 10.1007/s10107-022-01846-z. URL <https://doi.org/10.1007/s10107-022-01846-z>. Cited on page 3.
- [30] Francisco Facchinei and Vyacheslav Kungurtsev. Stochastic approximation for expectation objective and expectation inequality-constrained nonconvex optimization, 2023. URL <https://arxiv.org/abs/2307.02943>. Cited on pages 3 and 5.
- [31] Runchao Ma, Qihang Lin, and Tianbao Yang. Quadratically regularized subgradient methods for weakly convex optimization with weakly convex constraints. In *International Conference on Machine Learning*, pages 6554–6564. PMLR, 2020. Cited on page 3.
- [32] Frank E. Curtis, Daniel P. Robinson, and Baoyu Zhou. Sequential quadratic optimization for stochastic optimization with deterministic nonlinear inequality and equality constraints. *SIAM Journal on Optimization*, 34(4):3592–3622, 2024. doi: 10.1137/23M1556149. URL <https://doi.org/10.1137/23M1556149>. Cited on page 3.
- [33] Qiankun Shi, Xiao Wang, and Hao Wang. A momentum-based linearized augmented lagrangian method for nonconvex constrained stochastic optimization. *Optimization Online*, 2022. URL <https://optimization-online.org/?p=19870>. Cited on page 3.
- [34] Sen Na, Mihai Anitescu, and Mladen Kolar. Inequality constrained stochastic nonlinear optimization via active-set sequential quadratic programming, 2023. URL <https://arxiv.org/abs/2109.11502>. Cited on page 3.
- [35] Figen Oztoprak, Richard Byrd, and Jorge Nocedal. Constrained optimization in the presence of noise. *SIAM Journal on Optimization*, 33(3):2118–2136, 2023. doi: 10.1137/21M1450999. URL <https://doi.org/10.1137/21M1450999>. Cited on page 3.
- [36] Raghu Bollapragada, Cem Karamanli, Brendan Keith, Boyan Lazarov, Socratis Petrides, and Jingyi Wang. An adaptive sampling augmented lagrangian method for stochastic optimization with deterministic constraints. *Computers and Mathematics with Applications*, 149:239–258, 2023. ISSN 0898-1221. doi: <https://doi.org/10.1016/j.camwa.2023.09.014>. URL <https://www.sciencedirect.com/science/article/pii/S0898122123003991>. Cited on page 3.
- [37] Jose Gallego-Posada, Juan Ramirez, Meraj Hashemizadeh, and Simon Lacoste-Julien. Cooper: A Library for Constrained Optimization in Deep Learning, April 2025. Cited on page 3.
- [38] Andrii Kliachkin, Jana Lepšová, Gilles Bareilles, and Jakub Mareček. humancompatible.train: Implementing optimization algorithms for stochastically-constrained stochastic optimization problems. *NeurIPS Workshop on Constrained Optimization; arXiv preprint arXiv:2509.21254*, 2025. Cited on pages 3 and 15.
- [39] Boris T Polyak. A general method for solving extremal problems. In *Soviet Mathematics Doklady*, volume 8, pages 593–597, 1967. Cited on page 3.
- [40] Michal Kočvara and Michael Stingl. Pennon: A code for convex nonlinear and semidefinite programming. *Optimization methods and software*, 18(3):317–333, 2003. Cited on page 3.
- [41] Michal Kočvara and Michael Stingl. Pennon: a generalized augmented lagrangian method for semidefinite programming. In *High performance algorithms and software for nonlinear optimization*, pages 303–321. Springer, 2003. Cited on page 3.
- [42] Michal Kočvara and Michael Stingl. Pennon: Software for linear and nonlinear matrix inequalities. In *Handbook on semidefinite, conic and polynomial optimization*, pages 755–791. Springer, 2012. Cited on page 3.
- [43] Alberto De Marchi and Andreas Themelis. A penalty barrier framework for nonconvex constrained optimization. *Journal of Nonsmooth Analysis and Optimization*, 5(Original research articles), 2025. Cited on page 3.
- [44] R Tyrrell Rockafellar. Augmented lagrangians and applications of the proximal point algorithm in convex programming. *Mathematics of operations research*, 1(2):97–116, 1976. Cited on page 3.

- [45] R Tyrrell Rockafellar. Augmented lagrange multiplier functions and duality in nonconvex programming. *SIAM Journal on Control*, 12(2):268–285, 1974. Not cited.
- [46] R Tyrrell Rockafellar. The multiplier method of hestenes and powell applied to convex programming. *Journal of Optimization Theory and applications*, 12(6):555–562, 1973. Not cited.
- [47] R Tyrrell Rockafellar. A dual approach to solving nonlinear programming problems by unconstrained optimization. *Mathematical programming*, 5(1):354–373, 1973. Not cited.
- [48] Michael JD Powell. A method for nonlinear constraints in minimization problems. *Optimization*, pages 283–298, 1969. Not cited.
- [49] Magnus R Hestenes. Multiplier and gradient methods. *Journal of optimization theory and applications*, 4(5):303–320, 1969. Not cited.
- [50] Weiwei Kong, Jefferson G Melo, and Renato DC Monteiro. Complexity of a quadratic penalty accelerated inexact proximal point method for solving linearly constrained nonconvex composite programs. *SIAM Journal on Optimization*, 29(4):2566–2593, 2019. Not cited.
- [51] Weiwei Kong, Jefferson G Melo, and Renato DC Monteiro. Iteration complexity of a proximal augmented lagrangian method for solving nonconvex composite optimization problems with nonlinear convex constraints. *Mathematics of Operations Research*, 48(2):1066–1094, 2023. Not cited.
- [52] Wenqiang Pu, Kaizhao Sun, and Jiawei Zhang. Smoothed proximal lagrangian method for nonlinear constrained programs. *arXiv preprint arXiv:2408.15047*, 2024. Not cited.
- [53] Qihang Lin, Runchao Ma, and Yangyang Xu. Complexity of an inexact proximal-point penalty method for constrained smooth non-convex optimization. *Computational optimization and applications*, 82(1):175–224, 2022. Not cited.
- [54] Jiawei Zhang and Zhi-Quan Luo. A proximal alternating direction method of multiplier for linearly constrained nonconvex minimization. *SIAM Journal on Optimization*, 30(3):2272–2302, 2020. Cited on page 4.
- [55] Jiawei Zhang, Wenqiang Pu, and Zhi-Quan Luo. On the iteration complexity of smoothed proximal alm for nonconvex optimization problem with convex constraints. *arXiv preprint arXiv:2207.06304*, 2022. Cited on page 3.
- [56] Amir Beck. *First-Order Methods in Optimization*. SIAM, Philadelphia, PA, 2017. doi: 10.1137/1.9781611974997. Cited on page 4.
- [57] Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. Cited on page 5.
- [58] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations. *arXiv preprint arXiv:1711.10561*, 2017. Cited on pages 5 and 8.
- [59] Maarten Buyl, MaryBeth DeFrance, and Tijl De Bie. fairret: a framework for differentiable fairness regularization terms, 2024. URL <https://arxiv.org/abs/2310.17256>. Cited on page 6.
- [60] Sifan Wang, Xinling Yu, and Paris Perdikaris. When and why pinns fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, 2022. Cited on pages 6 and 7.
- [61] Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021. Cited on pages 6 and 7.

- [62] Levi D. McClenny and Ulisses M. Braga-Neto. Self-adaptive physics-informed neural networks. *Journal of Computational Physics*, 474:111722, February 2023. ISSN 0021-9991. doi: 10.1016/j.jcp.2022.111722. URL <http://dx.doi.org/10.1016/j.jcp.2022.111722>. Cited on page 14.
- [63] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. Cited on pages 14 and 25.

A Motivating example

In this section, we give details on the motivating example (Figure 1). We demonstrate the motivation for using SPBM over standard regularization through a constrained example with adversarial data. We consider a discrete random variable ξ such that $P(\xi = 0.1) = 0.5$ and $P(\xi = -0.1) = 0.5$ and we set the functions $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ and $g : \mathbb{R}^2 \times \Xi \rightarrow \mathbb{R}$ in (1) as

$$f(x) = x_1^2 + x_2^2, \quad g(x, \xi) = \min\{g_+(x, \xi), g_-(x, \xi)\},$$

where $g_{\pm}(x, \xi) = (x_1 \pm 2 + \xi)^2 + x_2^2 - 1$. The feasible region of problem (1) with this setting consists of two circles centered at $(-2, 0)$ and $(2, 0)$, each of radius $\sqrt{0.99}$. A standard solution is to consider the penalized objective F_{ρ} with a fixed, time-invariant regularization weight ρ :

$$F_{\rho}(x) = f(x) + \rho \|\mathbb{E}[g(x, \xi)]\|^2. \quad (16)$$

For large ρ , the stochastic gradient descent (SGD) applied to (16) could be unstable, whereas for too small ρ , SGD applied to (16) could be infeasible. In contrast, SPBM is stable and converges to the feasible set. This can be seen in Figure 1 comparing SGD on (16) with $\rho \in \{0.0, 1.0, 2.5\}$ and SPBM.

B PDEs used for PINNs Experiments

The Helmholtz equation reads:

$$\begin{aligned} \Delta u + u &= q(z_1, z_2), & \text{for every } (z_1, z_2) \in \Omega, \\ u(z_1, z_2) &= 0, & \text{for every } (z_1, z_2) \in \partial\Omega, \end{aligned}$$

where the function q is chosen as

$$\begin{aligned} q(z_1, z_2) &= -\pi^2 \sin(\pi z_1) \sin(4\pi z_2) \\ &\quad - (4\pi)^2 \sin(\pi z_1) \sin(4\pi z_2) + \sin(\pi z_1) \sin(4\pi z_2). \end{aligned}$$

The viscous Burgers equation reads:

$$\partial_t u + \partial_z \left(\frac{1}{2} u^2 - c \partial_z u \right) = 0, \quad \text{for } (t, z) \in [0, 1] \times [-1, 1], \quad (17)$$

$$u(0, z) = -\sin(\pi z), \quad \text{for } z \in [-1, 1], \quad (18)$$

$$u(t, -1) = u(t, 1) = 0, \quad \text{for } t \in [0, 1], \quad (19)$$

for $(t, z) \in [0, 1] \times [-1, 1]$, and $c = \frac{0.01}{\pi}$. This setting is the same as in [62].

C Experimental Setup

Architectures and epochs. To get a reliable estimate of the methods’ performance, we conduct three 30-epoch runs with three different seeds per each experiment for $(\mathbf{E}_1)$ – $(\mathbf{E}_6)$; for $(\mathbf{E}_7)$ and $(\mathbf{E}_8)$, we conduct three 6000-epoch runs with three different seeds. For experiment $(\mathbf{E}_1)$, we use an MLP with two hidden layers of size 64 and 32. For experiments $(\mathbf{E}_2)$ – $(\mathbf{E}_4)$, we use an MLP with 2 hidden layers with sizes 64 and 16 for $(\mathbf{E}_2)$ and $(\mathbf{E}_3)$, and with sizes 128 and 64 for $(\mathbf{E}_4)$. The experiment $(\mathbf{E}_5)$ is benchmarked using a convolutional neural network with two convolutional layers of respective depths 6 and 16 and a window size of 5, each followed by a 2×2 max-pooling layer, and three fully-connected layers. The experiment $(\mathbf{E}_6)$ is benchmarked using the ResNet-18 architecture [63]. For experiments $(\mathbf{E}_7)$ – $(\mathbf{E}_8)$, we use a deep narrow MLP with 8 hidden units of size 64.

Data split and processing. In all cases, we split the data into training, validation, and test sets using a 60/20/20 ratio representing the distributions $\mathcal{D}^{\text{train}}$, $\mathcal{D}^{\text{test}}$ and $\mathcal{D}^{\text{val}}$ respectively. All data of Experiments (E₁)–(E₆) were preprocessed to have zero mean and unit variance using the `scikit-learn` package. Data of Experiments (E₇)–(E₈) were not processed in any way.

The mini-batch size varied by problem: for experiments (E₁)–(E₄) the mini-batch sizes ranged from 30 to 72 samples, whereas for (E₅) and (E₆) the mini-batch sizes were 120 and 400, respectively. For experiments (E₇)–(E₈) the mini-batch sizes were set to 1000 samples.

Numerical setup. We run the experiments with AMD EPYC 9355 3.5GHz, 2TB RAM, Tesla H200 141GB NVLink (reference will be added for a camera-ready version).

Experiments compute resources for each experiment. We ran all of our experiments on HW specified in the previous paragraph.

The experiments (E₁)–(E₄): 1 minutes per run, we use 4x CPUs, 2GB of memory, we did not run these experiments on GPU.

The experiments (E₅)–(E₆): 2 minutes per run, 1x H200 GPU, 4x CPUs, 60GB of memory.

The experiments (E₇)–(E₈): 5 minutes per run, 1x H200, 4x CPUs, 1GB of memory.

Implementation Details The implementation of SSw and SSL-ALM are based on the repository Kliachkin et al. [38]. We base our implementation of the experiments described in “Solving PDEs as constrained optimization” in Section 4 on the code repository of Son et al. [8].

Choice of hyperparameters. To determine the hyperparameters, we performed a grid search for each algorithm averaging the results over three runs to account for random initialization. To transform inequality constraints into equalities, as required by SSL-ALM, we considered both introducing slack variables and taking $\max(g_i, 0)$ for each inequality constraint g_i ; we conducted a separate grid search for both options and reported the better-performing one in our plots. For experiments (E₁)–(E₄), we selected the combination of hyperparameters which yielded the lowest validation loss while keeping the validation constraints below the threshold plus a 10% tolerance. For experiments (E₅)–(E₆), we were unable to find a combination that would satisfy this requirement; instead, we chose those which minimize validation loss; For (E₇)–(E₈), we selected hyperparameters with the best validation loss, which also corresponded to the best constraint satisfaction. All final hyperparameters are listed in Tables 5 to 10 in Appendix D together with all grid search ranges.

D Hyperparameters

We list the final hyperparameters in Tables 5 to 12.

Table 5: (E₁) Weight regularization constraints, final hyperparameters.

	α	DUAL LR	ρ	PENALTY TYPE	μ	PENALTY UPDATE
ADAM	0.001	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
SSW	0.01	0.1	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
SSL-ALM	0.001	0.1	1.0	$\emptyset$	2.0	$\emptyset$
SPBM	0.001	0.95 (γ)	$\emptyset$	LOGARITHMIC	2.0	(8) WITH $K = 0.1$

Table 6: (E₂) ACSIncome, Equal Positive Rate, Manhattan norm of violations (L_1 norm), final hyperparameters.

	α	DUAL LR	ρ	PENALTY TYPE	μ	PENALTY UPDATE
ADAM	0.001	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
SSW	0.001	0.005	$\emptyset$	$\emptyset$	1.0	$\emptyset$
SSL-ALM	0.0001	0.001	1.0	$\emptyset$	1.0	$\emptyset$
SPBM	0.001	0.1 (γ)	$\emptyset$	LOGARITHMIC	2.0	(8) WITH $K = 0.999$

Table 7: (**E**₃) ACSIncome, Equal Positive Rate, Pairwise, final hyperparameters.

	α	DUAL LR	ρ	PENALTY TYPE	μ	PENALTY UPDATE
ADAM	0.001	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
SSW	0.001	0.0001	$\emptyset$	$\emptyset$	1.0	$\emptyset$
SSL-ALM	0.001	0.005	0.0	$\emptyset$	1.0	$\emptyset$
SPBM	0.0005	0.9 (γ)	$\emptyset$	LOGARITHMIC	1.0	(8) WITH $K = 0.999$

Table 8: (**E**₄) Dutch, Equal Positive Rate, Pairwise, final hyperparameters.

	α	DUAL LR	ρ	PENALTY TYPE	μ	PENALTY UPDATE
ADAM	0.001	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
SSW	0.005	0.005	$\emptyset$	$\emptyset$	1.0	$\emptyset$
SSL-ALM	0.001	0.005	0.0	$\emptyset$	1.0	$\emptyset$
SPBM	0.005	0.5 (γ)	$\emptyset$	LOGARITHMIC	1.0	(8) WITH $K = 0.8$

Table 9: (**E**₅) CIFAR-10, Equal Loss, Pairwise, final hyperparameters.

	α	DUAL LR	ρ	PENALTY TYPE	μ	PENALTY UPDATE
ADAM	0.001	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
SSW	0.01	0.001	$\emptyset$	$\emptyset$	1.0	$\emptyset$
SSL-ALM	0.001	0.001	0.0	$\emptyset$	1.0	$\emptyset$
SPBM	0.001	0.9 (γ)	$\emptyset$	RECIPROCAL	1.0	$p = 1$

Table 10: (**E**₆) CIFAR-100, Equal Loss, Pairwise, final hyperparameters.

	α	DUAL LR	ρ	PENALTY TYPE	μ	PENALTY UPDATE
ADAM	0.0001	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
SSW	0.01	0.1	$\emptyset$	$\emptyset$	1.0	$\emptyset$
SSL-ALM	0.001	0.001	0	$\emptyset$	1.0	$\emptyset$
SPBM	0.002	0.95 (γ)	$\emptyset$	RECIPROCAL	0.1	$p = 1$

Table 11: (**E**₇) Helmholtz PDE, final hyperparameters.

	α	DUAL LR	ρ	PENALTY TYPE	μ	PENALTY UPDATE	β
ADAM	0.001	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	5
SSL-ALM	0.001	0.005	1.0	$\emptyset$	1.0	$\emptyset$	$\emptyset$
SPBM	0.0005	0.2 (γ)	$\emptyset$	LOGARITHMIC	.0	(8) WITH $K = 0.99$	$\emptyset$

Table 12: (**E**₈) Viscous Burgers PDE, final hyperparameters.

	α	DUAL LR	ρ	PENALTY TYPE	μ	PENALTY UPDATE	β
ADAM	0.005	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	5
SSL-ALM	0.0005	0.01	1.0	$\emptyset$	1.0	$\emptyset$	$\emptyset$
SPBM	0.001	0.1 (γ)	$\emptyset$	LOGARITHMIC	.0	(8) WITH $K = 0.999$	$\emptyset$

D.1 Hyperparameter grid search

In all experiments, we use the PyTorch implementation of the Adam optimizer for the minimization of the objective (Adam, SSw) and Lagrangian (SSL-ALM, SPBM) functions. In all experiments except $(\mathbf{E}_1)$, we set `weight_decay=0.01` to provide slight L_2 weight regularization.

For experiments $(\mathbf{E}_1)$ through $(\mathbf{E}_4)$ an identical grid was employed.

- Adam: learning rate in $\{0.0001, 0.0005, 0.001, 0.005\}$
- SSw: $\{0.0001, 0.0005, 0.001, 0.005\}$ for objective and constraint learning rate
- SSL-ALM: $\{0.0001, 0.0005, 0.001, 0.005\}$ for objective and dual parameter learning rate; penalty multiplier ρ in $\{0, 1, 2\}$. To deal with inequality constraints, two approaches were tried: slack variables and thresholding the constraint against 0.
- SPBM: $\{0.0001, 0.0005, 0.001, 0.005\}$ for learning rate; γ in $\{0.1, 0.5, 0.9, 0.95, 0.99, 0.999, 0.9999\}$; K_1 in $\{0.99, 0.999, 0.9999, 1\}$ (for diminishing penalty update), and K_2 in $\{0.1, 0.8, 0.9, 0.99, 0.999\}$ (for adaptive penalty update). For these experiments, only the Quadratic-Logarithmic penalty/barrier function was used.
- Moreau envelope multiplier μ in $\{0.1, 1, 2\}$ for all methods; in case of SSw, applied to the objective.

For experiments $(\mathbf{E}_5)$, $(\mathbf{E}_6)$, the same grid as in $(\mathbf{E}_1)$ – $(\mathbf{E}_4)$ was used, but with the following changes:

- choice of objective learning rate for all methods changed to $\{0.0001, 0.001, 0.002, 0.005\}$;
- choice of the penalty multiplier ρ for the SSL-ALM reduced to $\{0, 1\}$ based on the results of previous experiments;
- for SPBM, choice between Quadratic-Logarithmic or Quadratic-Reciprocal penalty/barrier functions introduced.

For experiments $(\mathbf{E}_7)$, $(\mathbf{E}_8)$, the same grid as in $(\mathbf{E}_1)$ – $(\mathbf{E}_4)$ was used, but with the following changes:

- addition of the physics regularization parameter β for Adam in $\{0.1, 1, 2, 5\}$ for $(\mathbf{E}_7)$ and $\{1, 5, 10, 100\}$ for $(\mathbf{E}_8)$
- dual parameter learning rate increased to $\{0.0005, 0.001, 0.005, 0.01, 0.05\}$ for SSL-ALM.

E Remaining Results Figures

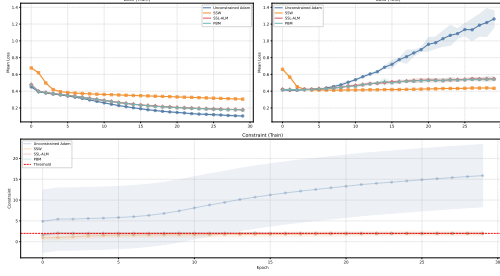


Figure 6: **(E₁): Weight regularization constraints, $m = 6$: mean loss** (top row: train and test) and **mean largest constraint** (bottom) values over 3 runs of 30 epochs of each method with random parameter initialization. The shaded region corresponds to ± 1 standard deviations. The red dotted line corresponds to the constraint threshold.

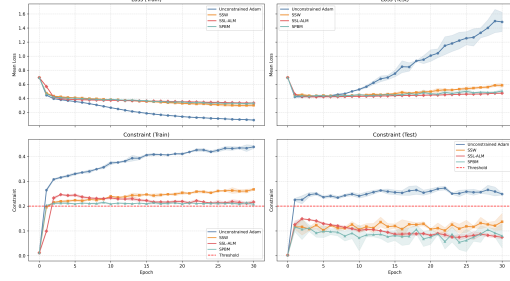


Figure 7: **(E₂): ACSIncome, Equal Accuracy, Manhattan norm of violations, $m = 1$: mean loss** (top row: train and test) and **mean constraint** (bottom row: train and test) values over 3 runs of 30 epochs of each method with random parameter initialization. The shaded region corresponds to ± 1 standard deviations. The red dotted line corresponds to the constraint threshold.

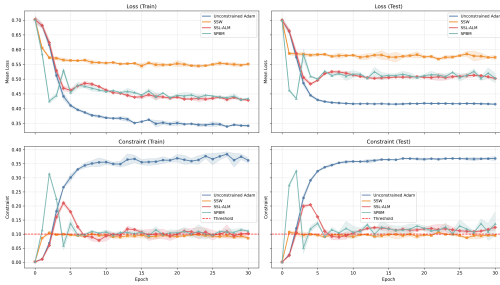


Figure 8: **(E₃): ACSIncome, Equal Accuracy, Pairwise, $m = 30$: mean loss** (top row: train and test) and **mean largest constraint** (bottom row: train and test) values over 30 epochs of each method with random parameter initialization. The shaded region corresponds to ± 1 standard deviations. The red dotted line corresponds to the constraint threshold.

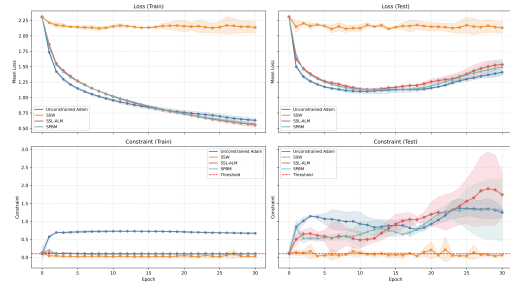


Figure 9: **(E₅): CIFAR-10, Equal Accuracy, Pairwise, $m = 90$: mean loss** (top row: train and test) and **mean largest constraint** (bottom row: train and test) values over 30 epochs of each method with random parameter initialization. The shaded region corresponds to ± 1 standard deviations. The red dotted line corresponds to the constraint threshold.

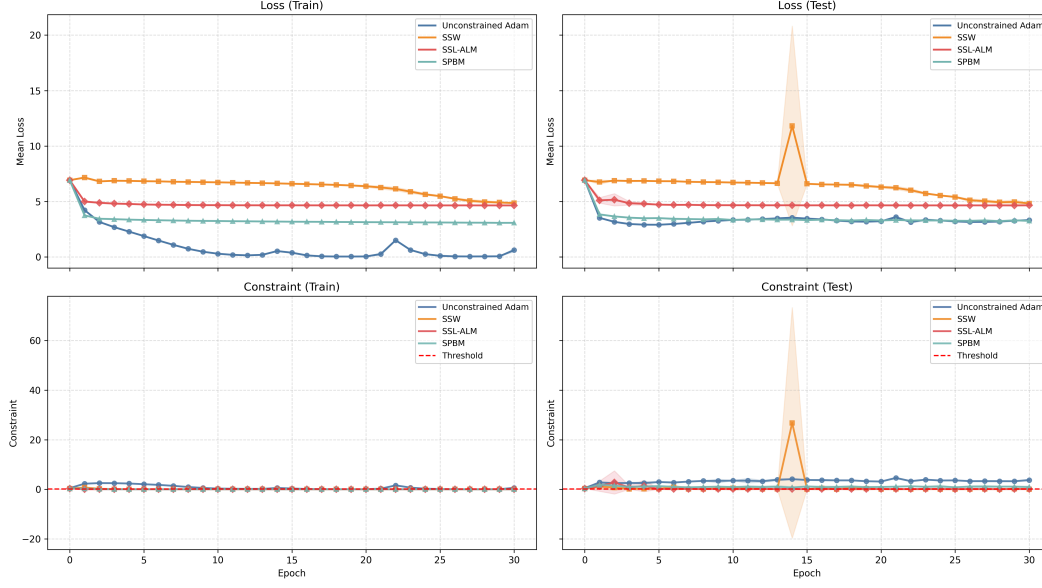


Figure 10: (E_6): CIFAR-100, Equal Accuracy, Pairwise, $m = 9900$: **mean loss** (top row: train and test) and **mean largest constraint** (bottom row: train and test) values over 3 runs of 30 epochs of each method with random parameter initialization. The shaded region corresponds to ± 1 standard deviations. The red dotted line corresponds to the constraint threshold. This is the second version of Fig. 3, where SSW is not included.

F List of Notation

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: We claim that our method matches or outperforms current existing methods for constrained non-smooth non-convex stochastic optimization. Such claim is reflected and supported by the experimental section 4 and by Table 4. Linear time overhead compared to unconstrained Adam is shown in ther Table 2.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: We acknowledge the limitations in the paragraph 6. Specifically, convergence guarantees in the non-convex stochastic setting remain an open problem in the field and are beyond the scope of this work. SPBM is evaluated on two problem classes (fairness-constrained classification and PINNs), and introduces additional hyperparameters compared to unconstrained baselines and the other two constrained methods.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The algorithm is described in detail in the section 3. Furthermore, in the section 4 and appendix C, we: (0) describe in detail the experiments we conducted (1) reference all the datasets needed to replicate our results, (2) describe and reference (if possible) all network architectures used, (3) describe data preprocessing, (4) describe how we chose the best hyperparameters for each method using the gridsearch (details D.1), and give tables with the best hyperparameters found (details D), (5) we specify hardware in paragraph C, (6) we also give our code as a supplementary material.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Yes, we provide code of our experiments with the description and README files.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.

- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
 - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
 - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
 - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
 - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
 - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
 - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.
6. **Experimental setting/details**
 Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?
 Answer: [Yes]
 Justification: Yes, we specify all the details about data, hyperparameters, and the rest of the experimental settings in sections 4 and C.
 Guidelines:
- The answer [N/A] means that the paper does not include experiments.
 - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
 - The full details can be provided either with the code, in appendix, or as supplemental material.
7. **Experiment statistical significance**
 Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?
 Answer: [Yes]
 Justification: All results are reported as mean $\pm$ standard deviation over 3 independent runs with different random seeds, capturing variability due to random initialization and stochastic mini-batching (Figures 2, 3, 4, 5, Tables 2, 4). Mean and standard deviation are computed using the standard unbiased estimator. The number of seeds is limited by computational budget. We specify all the details in the Experimental sections 4 and C.
 Guidelines:
- The answer [N/A] means that the paper does not include experiments.
 - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
 - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
 - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
 - The assumptions made should be given (e.g., Normally distributed errors).
 - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
 - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
 - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
 - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.
8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We specify all details about the needed time, compute and memory in the section C. Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: We have reviewed the NeurIPS Code of Ethics and the research abides by it.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: This work proposes a general constrained optimization method applicable to a range of settings, including fairness-aware classification. On the positive side, it enables practitioners to enforce fairness constraints during training, reducing discriminatory outcomes in deployed models. On the negative side, the flexibility of constraint specification could in principle be misused to encode constraints that entrench rather than mitigate bias, depending on how constraints are defined by the practitioner.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: [N/A]

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We give credit to baseline implementations in Section C. Models and datasets are cited in Sections 4 and C. Ding et al. [24] is used under the MIT License. The code of He et al. [63] and Kliachkin et al. [4] is used under the Apache 2.0 License. The Dutch demographic dataset [25] is copyright 2001 Centraal Bureau voor de Statistiek (Statistics Netherlands) and the Minnesota Population Center.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: We introduce code as a new asset, provided as part of the supplementary material as an anonymized zip file. The code is released under the Apache 2.0 license and includes a README with installation instructions and usage examples reproducing the experiments in the paper. No external datasets or models requiring consent were used.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLM usage was limited to editing and formatting.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.